অধ্যায় ৯

০৯

DOM Manipulation, Events এবং Interactive Web Applications

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- DOM কী, এবং Browser কীভাবে HTML থেকে Element-এর একটি গাছ তৈরি করে, তা ব্যাখ্যা করতে।
- Script কখন চলে তা নিয়ন্ত্রণ করতে, এবং `null` Element-এর Error এড়াতে।
- `querySelector` ও `querySelectorAll` দিয়ে Element খুঁজতে, পড়তে এবং বদলাতে।
- `createElement`, `append`, `remove` ও `replaceChildren` দিয়ে Page-এর অংশ তৈরি ও সরাতে।
- Event Listener যোগ করতে, Event Object পড়তে এবং Event Propagation বুঝতে।
- Event Delegation দিয়ে একটি Listener-এ অনেক Element, এমনকি পরে তৈরি হওয়া Element-ও সামলাতে।
- Form-এর Submit সামলাতে এবং ব্যবহারকারীকে বোধগম্যভাবে ভুল জানাতে।
- State ও UI আলাদা রেখে একটি `render` Function-এর মাধ্যমে Page আঁকতে।
- `localStorage`-এ তথ্য নিরাপদে রাখতে ও ফিরিয়ে আনতে।
- একটি সম্পূর্ণ **Task Manager** Web Application তৈরি করতে এবং Browser-এ Automated Test দিয়ে পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৭: Object, Spread এবং `JSON.stringify` / `JSON.parse`।
- অধ্যায় ৮: `textContent` বনাম `innerHTML` এবং XSS।
- HTML-এর প্রাথমিক ধারণা: Tag, `id`, `class`, `<form>`, `<input>`, `<button>`।
- একটি আধুনিক Browser (Chrome, Edge বা Firefox) এবং তার DevTools।
- `chapter-09` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত আপনার Program-গুলো Console-এ লেখা দেখিয়েছে। কিন্তু ব্যবহারকারী Console দেখেন না; তাঁরা দেখেন একটি Page, যেখানে Button চাপেন, লেখেন, তালিকা থেকে বাছেন। JavaScript-কে Browser-এ তৈরি করার মূল কারণ ছিল এটিই: Page-কে জীবন্ত করা।

এই অধ্যায়ে আপনি শিখবেন Page পড়তে ও বদলাতে, এবং ব্যবহারকারীর প্রতিটি কাজে সাড়া দিতে। সঙ্গে শিখবেন এমন একটি গঠন, যা Application বড় হলেও Code-কে গোছানো রাখে: তথ্য এক জায়গায়, আঁকার Code আরেক জায়গায়। React, Vue বা Svelte-এর মতো আধুনিক Framework এই ধারণার ওপরই তৈরি। ধারণাটি এখানে খালি হাতে বুঝে নিলে পরে সেগুলো শেখা অনেক সহজ হবে।

৯.১ DOM: Page-এর গাছ

Browser একটি HTML File পড়ে তার প্রতিটি Tag থেকে একটি করে **Element Object** তৈরি করে, এবং সেগুলোকে মূল HTML-এর মতোই একটির ভেতরে আরেকটি সাজিয়ে রাখে। এই Object-এর গাছকে বলে **DOM** (Document Object Model)। JavaScript এই গাছ পড়তে ও বদলাতে পারে, আর গাছ বদলালে Browser সঙ্গে সঙ্গে পর্দার ছবি বদলে দেয়।

নিচের Page-টিতে একটি শিরোনাম এবং একটি তালিকা আছে।

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>DOM পড়া</title>
</head>
<body>
  <h1 id="title">আজকের কাজ</h1>
  <ul class="tasks">
    <li data-priority="high">বাজার করা</li>
    <li>বই পড়া</li>
    <li class="done">হাঁটা</li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

এর DOM গাছটি এমন: সবার ওপরে `document`, তার নিচে `html`, তার দুটি শাখা `head` ও `body`, এবং `body`-র ভেতরে `h1`, `ul` ও `script`। `ul`-এর তিনটি সন্তান (Child) `li`। গাছের ভাষায় `ul` হলো `li`-গুলোর **Parent**, আর `li`-গুলো পরস্পরের **Sibling**।

Element খোঁজা ও পড়া

script.js · JavaScript চলবে: Browser

```
const title = document.querySelector('#title');
const items = document.querySelectorAll('.tasks li');

console.log(title.textContent);
console.log(items.length);
console.log(items[0].dataset.priority);
console.log(document.querySelector('.done').textContent);
console.log(document.querySelector('#missing'));
console.log([...items].map((item) => item.textContent).join(', '));
```

Output

```
আজকের কাজ
3
high
হাঁটা
null
বাজার করা, বই পড়া, হাঁটা
```

- **`document.querySelector(selector)`** CSS-এর Selector লিখে প্রথম মিলে যাওয়া Element দেয়: `#title` মানে `id="title"`, `.done` মানে `class="done"`, `.tasks li` মানে `.tasks`-এর ভেতরের সব `li`। কিছু না মিললে `null`।
- **`document.querySelectorAll(selector)`** সব মিল একটি তালিকায় দেয় (`NodeList`)। এর `length` আছে, Index দিয়ে পড়া যায়, `for...of` চলে; কিন্তু `map`-এর মতো Array Method নেই। তাই শেষ লাইনে Spread দিয়ে আগে Array বানানো হয়েছে (অধ্যায় ৭)।
- **`dataset`** দিয়ে `data-` দিয়ে শুরু হওয়া Attribute পড়া যায়: `data-priority` হয় `dataset.priority`। নিজের তথ্য Element-এর সঙ্গে জুড়ে রাখার এটিই মানসম্মত উপায়।

পুরোনো Code-এ `getElementById`, `getElementsByClassName` দেখবেন। সেগুলো এখনো কাজ করে, তবে `querySelector` একাই সব ধরনের Selector বোঝে, তাই এই বইয়ে সেটিই ব্যবহার করা হয়েছে।

৯.২ Script কখন চলে

Browser HTML ওপর থেকে নিচে পড়ে। পথে `<script>` পেলে সেখানেই থেমে Script চালায়, তারপর বাকি HTML পড়ে। ফলে Script যদি এমন Element খোঁজে যা তখনো পড়াই হয়নি, তা পাবে না।

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>খুব আগে</title>
  <script src="script.js"></script>
</head>
<body>
  <h1 id="title">স্বাগতম</h1>
</body>
</html>
```

script.js · JavaScript চলবে: Browser

```
const title = document.querySelector('#title');

console.log(title);
title.textContent = 'নতুন শিরোনাম';
```

Error Message

```
null
TypeError: Cannot set properties of null (setting 'textContent')
```

Script চলার সময় `<h1>` তখনো তৈরি হয়নি, তাই `querySelector` দিয়েছে `null`, এবং `null`-এর ওপর `textContent` বসাতে গিয়ে Error। DOM নিয়ে কাজের সবচেয়ে প্রচলিত Error এটিই। সমাধান দুটি:

১. **`<script>`-কে `<body>`-র একেবারে শেষে রাখুন,** যেমন অনুচ্ছেদ ৯.১-এর Page-এ।

২. **`<head>`-এ রাখলে `defer` লিখুন:** `<script src="script.js" defer></script>`। `defer` থাকলে Browser File-টি সঙ্গে সঙ্গে আনা শুরু করে, কিন্তু চালায় পুরো HTML পড়া শেষ হওয়ার পর।

একাধিক `defer` Script লেখা ক্রমেই চলে। এই অধ্যায়ের Project প্রথম পদ্ধতি ব্যবহার করেছে।

৯.৩ Element বদলানো

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>বদলানো</title>
</head>
<body>
  <p id="message" class="note">অপেক্ষা করছে</p>
  <a id="help" href="https://example.com">সহায়তা</a>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```
const message = document.querySelector('#message');
const help = document.querySelector('#help');

message.textContent = 'সংরক্ষিত হয়েছে';
message.classList.add('success');
message.classList.remove('note');
message.classList.toggle('bold');

help.setAttribute('href', 'https://developer.mozilla.org');
help.hidden = true;

console.log(message.textContent);
console.log(message.className);
console.log(message.classList.contains('note'));
console.log(help.getAttribute('href'));
console.log(help.hidden);
```

Output

```
সংরক্ষিত হয়েছে
success bold
false
https://developer.mozilla.org
true
```

কাজ	উপায়
লেখা বদলানো	`element.textContent = text`

কাজ	উপায়
Class যোগ / সরানো / উল্টানো	`classList.add()`, `remove()`, `toggle()`
Class আছে কি না	`classList.contains()`
Attribute পড়া / বসানো / সরানো	`getAttribute()`, `setAttribute()`, `removeAttribute()`
লুকানো / দেখানো	`element.hidden = true` বা `false`
Form Field-এর মান	`input.value`, `checkbox.checked`

চেহারা বদলাতে সরাসরি `element.style.color = 'red'` লেখা যায়, কিন্তু ভালো অভ্যাস হলো CSS-এ একটি Class-এর চেহারা লিখে রাখা এবং JavaScript থেকে শুধু Class যোগ বা সরানো। তাতে চেহারার সব নিয়ম CSS-এ এক জায়গায় থাকে।

> **সতর্কতা:** লেখা বসাতে সব সময় `textContent`। `innerHTML` লেখাকে HTML হিসেবে পড়ে; ব্যবহারকারীর লেখা সেখানে গেলে XSS-এর পথ খুলে যায় (অধ্যায় ৮)।

৯.৪ Element তৈরি, যোগ ও সরানো

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>তৈরি ও সরানো</title>
</head>
<body>
  <ul id="list">
    <li>প্রথম</li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```javascript
const list = document.querySelector('#list');

const second = document.createElement('li');
second.textContent = 'দ্বিতীয়';
list.append(second);

const first = document.createElement('li');
```

```
first.textContent = 'শুরুতে';
list.prepend(first);

console.log(list.children.length);

list.firstElementChild.remove();
console.log(list.children.length);
console.log(list.lastElementChild.textContent);

list.replaceChildren();
console.log(list.children.length);
```

Output

```
3
2
দ্বিতীয়
0
```

নতুন Element তৈরির তিন ধাপ: `document.createElement('tag')` দিয়ে তৈরি, Property বসানো, তারপর Page-এর কোথাও যোগ। তৈরি করা Element যোগ না করা পর্যন্ত Page-এ দেখা যায় না।

Method	কাজ
`parent.append(a, b)`	শেষে যোগ; একসঙ্গে একাধিক
`parent.prepend(a)`	শুরুতে যোগ
`element.remove()`	নিজেকে Page থেকে সরানো
`parent.replaceChildren(a, b)`	সব পুরোনো সন্তান সরিয়ে নতুনগুলো বসানো; কিছু না দিলে খালি করা
`parent.children`	সন্তান Element-গুলোর তালিকা

অনেক Element একসঙ্গে বসাতে হলে প্রথমে `map` দিয়ে সবগুলো তৈরি করুন, তারপর একবারে `replaceChildren(...elements)` বা `append(...elements)`। এতে Browser-কে একবারই পর্দা নতুন করে আঁকতে হয়, এবং Code-ও পরিষ্কার থাকে।

৯.৫ Event: ব্যবহারকারীর কাজে সাড়া

ব্যবহারকারী Click করলে, লিখলে, Key চাপলে বা Form পাঠালে Browser একটি **Event** তৈরি করে। আপনি আগে থেকে বলে রাখতে পারেন, অমুক Element-এ অমুক Event ঘটলে এই Function চালাতে হবে। এই Function-কে বলে **Event Listener** বা Handler।

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>গণনা</title>
</head>
<body>
  <p>গণনা: <output id="count">0</output></p>
  <button id="add" type="button">যোগ</button>
  <button id="reset" type="button">শূন্য</button>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript চলবে: Browser

```javascript
const countView = document.querySelector('#count');
let count = 0;

function show() {
  countView.textContent = count;
  console.log(`গণনা: ${count}`);
}

document.querySelector('#add').addEventListener('click', () => {
  count += 1;
  show();
});

document.querySelector('#reset').addEventListener('click', () => {
  count = 0;
  show();
});
```

Browser-এ Page খুলে "যোগ", "যোগ", "শূন্য", "যোগ" চাপলে:

Output

```
গণনা: 1
গণনা: 2
গণনা: 0
গণনা: 1
```

`addEventListener('click', function)`-এ দ্বিতীয় Argument হলো Function নিজে, যা Browser পরে Call করবে। Page Load-এর সময় কিছুই ছাপা হয়নি; প্রতিটি লাইন একটি Click-এর ফল।

Event	কখন ঘটে
`click`	Click, অথবা Button-এ Focus রেখে Enter বা Space
`input`	Field-এর লেখা প্রতিবার বদলালে
`change`	Checkbox, Select বদলালে; Text Field-এ লেখা শেষ করে বেরোলে
`submit`	Form পাঠানো হলে (Button বা Enter)
`keydown`	Key চাপলে; কোন Key তা `event.key`-এ
`DOMContentLoaded`	পুরো HTML পড়া শেষ হলে

> **মনে রাখুন:** `button.addEventListener('click', show())` লিখলে `show` সঙ্গে সঙ্গে একবার চলে, এবং তার ফেরত মান (`undefined`) Listener হিসেবে যায়। ফলে Click-এ কিছুই হয় না। Function-এর নাম দিন, Call নয়: `show`।

৯.৬ Event Object এবং Propagation

Listener Function একটি Argument পায়: **Event Object**। এতে Event-এর সব তথ্য থাকে: কোন ধরনের Event (`event.type`), কোন Element-এ ঘটেছে (`event.target`), কোন Key (`event.key`)।

একটি গুরুত্বপূর্ণ আচরণ: একটি Element-এ Event ঘটলে তা ওই Element-এ শেষ হয় না; DOM গাছ বেয়ে Parent, তার Parent, এভাবে `document` পর্যন্ত ওপরে ওঠে। একে বলে **Bubbling**।

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Bubbling</title>
</head>
<body>
  <div id="card">
    <h2>কার্ড</h2>
    <button id="like" type="button">পছন্দ</button>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```
const card = document.querySelector('#card');
const like = document.querySelector('#like');

like.addEventListener('click', (event) => {
  console.log(`like: ${event.type}`);
});

card.addEventListener('click', (event) => {
  console.log(`card: target ${event.target.tagName}`);
});

document.addEventListener('click', () => {
  console.log('document');
});
```

প্রথমে "পছন্দ" Button, তারপর "কার্ড" শিরোনামে Click করলে:

Output

```
like: click
card: target BUTTON
document
card: target H2
document
```

Button-এ একটি Click তিনটি Listener চালিয়েছে: Button-এর, তার Parent `card`-এর, এবং `document`-এর। `card`-এর Listener-এ `event.target` হলো আসল Click যেখানে হয়েছে সেই Element: প্রথমবার Button, দ্বিতীয়বার শিরোনাম।

- **`event.stopPropagation()`** Event-কে আর ওপরে উঠতে দেয় না। এটি খুব কম প্রয়োজন হয়, এবং অন্য Code-এর প্রত্যাশা ভেঙে দিতে পারে; প্রয়োজন না হলে ব্যবহার করবেন না।
- **`event.preventDefault()`** Browser-এর নিজস্ব আচরণ থামায়: Link-এ গিয়ে Page বদলানো, বা Form পাঠিয়ে Page নতুন করে Load। এটি Propagation থামায় না; দুটি আলাদা জিনিস।

৯.৭ Event Delegation

Bubbling-এর সবচেয়ে বড় ব্যবহার **Event Delegation**: প্রতিটি সন্তানে আলাদা Listener না দিয়ে Parent-এ একটি Listener দেওয়া, এবং `event.target` দেখে বোঝা কোন সন্তানে ঘটেছে।

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Delegation</title>
</head>
<body>
  <ul id="menu">
    <li><button type="button" data-item="চা">চা</button></li>
    <li><button type="button" data-item="কফি">কফি</button></li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```javascript
const menu = document.querySelector('#menu');

menu.addEventListener('click', (event) => {
  const button = event.target.closest('button');

  if (!button) {
    return;
  }

  console.log(`অর্ডার: ${button.dataset.item}`);
});

const row = document.createElement('li');
const juice = document.createElement('button');
juice.type = 'button';
juice.dataset.item = 'জুস';
juice.textContent = 'জুস';
row.append(juice);
menu.append(row);
```

"চা" এবং পরে যোগ হওয়া "জুস" Button-এ Click করলে:

Output

```
অর্ডার: চা
অর্ডার: জুস
```

"জুস" Button Listener যোগ হওয়ার **পরে** তৈরি হয়েছে, তবু কাজ করেছে, কারণ Listener আছে Parent-এ। তালিকা যেখানে বারবার নতুন করে আঁকা হয়, সেখানে এটি অপরিহার্য: প্রতিবার আঁকার পর Listener নতুন করে যোগ করতে হয় না।

‘‘

`event.target.closest('button')` Click-এর জায়গা থেকে ওপরের দিকে প্রথম `button` খোঁজে, নিজেকেসহ। Button-এর ভেতরে একটি Icon বা `<span>` থাকলে `event.target` সেই ভেতরের Element হতে পারে; `closest` তখনো আসল Button-টিই দেয়। তালিকার ফাঁকা জায়গায় Click হলে `null`, এবং Guard Clause (অধ্যায় ৫) চুপচাপ ফিরে যায়।

৯.৮ Form এবং Validation

Form-এর তথ্য পড়ার সবচেয়ে নির্ভরযোগ্য জায়গা `submit` Event। এটি Button-এ Click এবং Field-এ Enter চাপা—দুই ক্ষেত্রেই ঘটে।

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Form</title>
</head>
<body>
  <form id="signup" novalidate>
    <label for="email">Email</label>
    <input id="email" type="email" aria-describedby="error">
    <button type="submit">নিবন্ধন</button>
  </form>
  <p id="error" role="alert"></p>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript চলবে: Browser

```javascript
const form = document.querySelector('#signup');
const email = document.querySelector('#email');
const error = document.querySelector('#error');

form.addEventListener('submit', (event) => {
  event.preventDefault();
  const value = email.value.trim();

  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)) {
    error.textContent = 'সঠিক একটি Email লিখুন।';
    email.setAttribute('aria-invalid', 'true');
    console.log(`ত্রুটি: "${value}"`);
    return;
  }
```

```
  error.textContent = '';
  email.removeAttribute('aria-invalid');
  console.log(`পাঠানো হলো: ${value}`);
  form.reset();
});
```

ফাঁকা রেখে "নিবন্ধন" চাপলে, তারপর `sara@` লিখে Enter, শেষে `sara@example.com` লিখে Enter:

Output

```
ত্রুটি: ""
ত্রুটি: "sara@"
পাঠানো হলো: sara@example.com
```

- **`event.preventDefault()`** প্রথম লাইনেই। এটি না থাকলে Browser Form পাঠিয়ে Page নতুন করে Load করত, এবং আপনার Code-এর সব কাজ ও Console মুছে যেত।
- **`novalidate`** Browser-এর নিজস্ব যাচাই-বার্তা বন্ধ করেছে, যাতে বার্তা ও তার ভাষা আপনার নিয়ন্ত্রণে থাকে। Browser-এর যাচাই চালু রাখাও একটি বৈধ পছন্দ; দুটি একসঙ্গে মেশাবেন না।
- **যাচাই Browser-এ, আবার Server-এও।** Browser-এর যাচাই ব্যবহারকারীর সুবিধার জন্য; যেকোনো ব্যবহারকারী তা এড়িয়ে যেতে পারেন। আসল নিরাপত্তা Server-এ।

সবার জন্য ব্যবহারযোগ্য Form

Screen Reader ব্যবহারকারী পর্দা দেখেন না; তাঁদের Software পড়ে শোনায়। কয়েকটি ছোট অভ্যাস তাঁদের জন্য Form ব্যবহারযোগ্য করে:

- প্রতিটি Field-এর একটি `<label>`, যার `for` Field-এর `id`-এর সঙ্গে মেলে।
- ভুলের বার্তা `role="alert"` বা `aria-live="polite"` আছে এমন Element-এ; লেখা বদলালে Software তা পড়ে শোনায়।
- ভুল Field-এ `aria-invalid="true"`, এবং `aria-describedby` দিয়ে বার্তার সঙ্গে সংযোগ।
- সব কাজ Keyboard দিয়ে করা যায় কি না যাচাই: Tab দিয়ে ঘোরা, Enter দিয়ে পাঠানো। আসল `<button>` ও `<form>` ব্যবহার করলে এটি বিনা খরচে পাওয়া যায়।

৯.৯ State এবং UI আলাদা রাখা

ছোট Page-এ সরাসরি DOM বদলানো চলে। কিন্তু Page বড় হলে প্রশ্ন ওঠে: আসল তথ্য কোথায়? Page-এর লেখায়, নাকি কোনো Variable-এ? দুই জায়গায় থাকলে একসময় দুটি আলাদা হয়ে যায়।

নিয়মটি সরল: **তথ্য থাকবে একটি জায়গায়, যাকে বলে State। Page হবে সেই State-এর ছবি।** তথ্য বদলাতে হলে State বদলান, তারপর একটি `render` Function পুরো অংশটি State থেকে আবার আঁকে।

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>State</title>
</head>
<body>
  <ul id="cart"></ul>
  <p id="total"></p>
  <button id="add-tea" type="button">চা যোগ</button>
  <script src="script.js"></script>
</body>
</html>
```

script.js · JavaScript — চলবে: Browser

```javascript
const state = { items: [{ name: 'বই', price: 250 }] };

const cartView = document.querySelector('#cart');
const totalView = document.querySelector('#total');

function render() {
  const rows = state.items.map((item) => {
    const row = document.createElement('li');
    row.textContent = `${item.name}: ${item.price} টাকা`;
    return row;
  });
  const total = state.items.reduce((sum, item) => sum + item.price, 0);

  cartView.replaceChildren(...rows);
  totalView.textContent = `মোট ${total} টাকা`;
  console.log(`${rows.length}টি জিনিস, ${totalView.textContent}`);
}

document.querySelector('#add-tea').addEventListener('click', () => {
  state.items = [...state.items, { name: 'চা', price: 20 }];
  render();
});

render();
```

Page খোলার পর "চা যোগ" দুবার চাপলে:

Output

```
1টি জিনিস, মোট 250 টাকা
2টি জিনিস, মোট 270 টাকা
3টি জিনিস, মোট 290 টাকা
```

Click Handler পর্দার কিছু ছোঁয়নি; শুধু State বদলে `render()` ডেকেছে। মোট দাম কোথাও আলাদাভাবে যোগ-বিয়োগ করে রাখা হয়নি; প্রতিবার State থেকে নতুন করে হিসাব হয়েছে, তাই ভুল হওয়ার সুযোগ নেই।

এর সঙ্গে আরেকটি বিভাজন যোগ করুন: তথ্য বদলানোর নিয়ম (Task যোগ, মোছা, যাচাই) রাখুন DOM-মুক্ত Pure Function-এ (অধ্যায় ৫)। তখন সেই নিয়মগুলো Browser ছাড়াই Node.js-এ Test করা যায়। Project ঠিক এভাবেই গঠিত।

৯.১০ localStorage: নিরাপদে সংরক্ষণ

অধ্যায় ৭-এ দেখেছেন, `localStorage` Browser-এ লেখা আকারে তথ্য রাখে, এবং Object রাখতে `JSON.stringify` লাগে। এখানে তিনটি বাড়তি বিষয় দেখা যাক।

storage.js · JavaScript — চলবে: Browser

```
const KEY = 'jsm-ch09-settings';

localStorage.setItem(KEY, JSON.stringify({ theme: 'dark', fontSize: 16 }));

const raw = localStorage.getItem(KEY);
console.log(typeof raw);
console.log(JSON.parse(raw).fontSize + 2);

localStorage.setItem('count', 5);
console.log(typeof localStorage.getItem('count'));
console.log(localStorage.getItem('missing'));

localStorage.removeItem(KEY);
localStorage.removeItem('count');
console.log(localStorage.length);
```

Output

```
string
18
string
null
0
```

১. **সব কিছু লেখা হয়ে যায়।** `5` রাখলে ফেরত আসে `'5'`। সংখ্যা, Boolean, Object—সবই `JSON.stringify` দিয়ে রাখুন এবং `JSON.parse` দিয়ে ফিরিয়ে আনুন।

২. **না থাকলে `null`।** প্রথমবার Page খোলার সময় এটিই স্বাভাবিক অবস্থা।

৩. **সংরক্ষিত তথ্য বিশ্বাসযোগ্য নয়।** পুরোনো Version-এর Program অন্য গঠনে রেখে থাকতে পারে, ব্যবহারকারী DevTools-এ বদলে থাকতে পারেন, অথবা Browser Storage বন্ধ রাখতে পারে। তাই পড়ার Code কখনো ভেঙে পড়া উচিত নয়।

safe-storage.js · JavaScript — চলবে: Browser

```javascript
function loadList(key) {
  try {
    const data = JSON.parse(localStorage.getItem(key) ?? '[]');
    return Array.isArray(data) ? data : [];
  } catch {
    return [];
  }
}

localStorage.setItem('broken', '{not json');
localStorage.setItem('wrong-shape', '{"a":1}');

console.log(loadList('broken').length);
console.log(loadList('wrong-shape').length);
console.log(loadList('never-saved').length);

localStorage.removeItem('broken');
localStorage.removeItem('wrong-shape');
```

Output

```
0
0
0
```

ভাঙা JSON, ভুল গঠন, বা কিছুই না থাকা—তিন ক্ষেত্রেই Program একটি ফাঁকা তালিকা নিয়ে স্বাভাবিকভাবে শুরু করেছে। `try...catch` Error ধরে Program-কে চালু রাখে; বিস্তারিত অধ্যায় ১১-এ। `setItem`-ও ব্যর্থ হতে পারে (জায়গা শেষ, বা Storage বন্ধ), তাই সেটিও `try...catch`-এ রাখুন।

> **সতর্কতা:** `localStorage` একই Website-এর যেকোনো Script পড়তে পারে, এবং তা Encrypt করা থাকে না। Password, Token বা ব্যক্তিগত সংবেদনশীল তথ্য সেখানে রাখবেন না।

৯.১১ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Element তৈরির আগে Script চলা	`Cannot set properties of null`	Script `<body>` -র শেষে, বা `defer`
ভুল Selector (`title` বনাম `#title`)	`null`	DevTools Console-এ Selector পরীক্ষা
`addEventListener('click', show())`	Load-এ একবার চলে, Click-এ কিছু না	`show` (Call ছাড়া)
Submit Handler-এ `preventDefault` নেই	Page Reload, তথ্য হারায়	প্রথম লাইনেই `event.preventDefault()`
পরে তৈরি Element-এ Listener নেই	নতুন Button কাজ করে না	Event Delegation
`event.target` সরাসরি Button ধরা	ভেতরের `<span>` -এ Click হলে কাজ করে না	`event.target.closest('button')`
`querySelectorAll` -এর ওপর `map`	`map is not a function`	`[...nodeList].map()`
ব্যবহারকারীর লেখা `innerHTML` -এ	XSS	`textContent`
`localStorage` থেকে পড়া তথ্য যাচাই না করা	ভাঙা তথ্যে পুরো Page বন্ধ	`try...catch` এবং গঠন যাচাই
State ও DOM দুই জায়গায় তথ্য	পর্দা ও তথ্য মেলে না	এক State, এক `render`

৯.১২ Debugging নির্দেশিকা

Browser-এর DevTools (F12, অথবা Page-এ Right Click করে Inspect) DOM নিয়ে কাজের প্রধান যন্ত্র।

- **Console:** Error-এর লাল বার্তা এবং তার পাশে File ও লাইন নম্বর। লাইনে Click করলে সেই Code খোলে। `document.querySelector('#title')` সরাসরি লিখে দেখুন Selector কিছু পায় কি না।
- **Elements:** বর্তমান DOM গাছ, মূল HTML নয়। JavaScript যা বদলেছে, এখানে তা দেখা যায়। একটি Element বেছে নিয়ে Console-এ `$0` লিখলে সেটি পাওয়া যায়।
- **Event Listeners (Elements-এর পাশের Panel):** কোন Element-এ কোন Listener লাগানো আছে। Listener না থাকলে, বা ভুল Element-এ থাকলে এখানে ধরা পড়ে।

- **Sources:** Code-এর লাইনে Breakpoint দিয়ে Program থামানো, এবং সেই মুহূর্তের Variable দেখা। Event Listener-এর ভেতরে Breakpoint দিলে Click-এর সময় থামে।
- **Application:** Local Storage-এ কী সংরক্ষিত আছে দেখা, বদলানো ও মোছা। "সংরক্ষণ কাজ করছে না" ধরনের সমস্যায় প্রথম জায়গা।

একটি Click কাজ না করলে ক্রমে দেখুন:

১. Console-এ কোনো Error আছে কি না; থাকলে প্রথম Error থেকে শুরু করুন।
২. Selector সঠিক Element পায় কি না (`null` নয় তো)।
৩. Listener সঠিক Element-এ লাগানো আছে কি না (Event Listeners Panel)।
৪. Listener-এর প্রথম লাইনে `console.log('click')` দিয়ে দেখুন আদৌ চলে কি না।
৫. চললে State ঠিকমতো বদলেছে কি না, তারপর `render` ডাকা হয়েছে কি না।

৯.১৩ পেশাদার অভ্যাস

- **তথ্যের নিয়ম DOM থেকে আলাদা File-এ,** এবং সেগুলো Pure Function। তাতে Test সহজ হয়।
- **একটি State, একটি `render`।** Event Handler State বদলায়; পর্দা আঁকে শুধু `render`।
- **তালিকায় Event Delegation;** Element-এ নিজের তথ্য রাখতে `data-` Attribute।
- **লেখা সব সময় `textContent` দিয়ে।**
- **আসল HTML Element ব্যবহার করুন:** Click-যোগ্য জিনিস `<button>`, Form `<form>`, প্রতিটি Field-এর `<label>`। `<div>`-এ Click Listener দিলে Keyboard ও Screen Reader ব্যবহারকারী বাদ পড়েন।
- **`<button type="button">`** লিখুন যদি Button Form পাঠানোর জন্য না হয়; Form-এর ভেতরে Button-এর স্বাভাবিক ধরন `submit`।
- **ব্যবহারকারীকে সব সময় জানান** কী হলো: সফল হলে ছোট বার্তা, ভুল হলে কী ঠিক করতে হবে।
- **সংরক্ষিত তথ্যের Key-তে নাম ও Version** (`'jsm-ch09-tasks'`), যাতে অন্য Program-এর সঙ্গে মিশে না যায়।

৯.১৪ Project: Task Manager

একটি সম্পূর্ণ Web Application: কাজের তালিকা, যেখানে Task যোগ, সম্পন্ন চিহ্নিত, নাম বদলানো, মোছা, খোঁজা ও ছাঁকা যায়, এবং Page বন্ধ করে খুললেও তালিকা থাকে।

Project-এর চাহিদা

- Task যোগ: ফাঁকা নাম গ্রহণযোগ্য নয়; সর্বোচ্চ ১২০ অক্ষর; Enter দিয়েও যোগ করা যাবে।
- Checkbox দিয়ে সম্পন্ন বা বাকি চিহ্নিত করা।
- প্রতিটি Task-এর নাম বদলানো (Enter-এ সংরক্ষণ, Escape-এ বাতিল) এবং মোছা।

- নাম দিয়ে খোঁজা এবং "সব / বাকি / সম্পন্ন" ছাঁকা; সম্পন্নগুলো একসঙ্গে সরানো।
- মোট, বাকি ও সম্পন্ন Task-এর সংখ্যা।
- `localStorage`-এ সংরক্ষণ; ভাঙা সংরক্ষিত তথ্যেও Page চালু থাকবে।
- নিরাপত্তা: Task-এর নাম শুধু `textContent` দিয়ে দেখানো।
- সবার জন্য ব্যবহারযোগ্য: Label, Keyboard দিয়ে সব কাজ, বার্তা `aria-live`-এ।

File-এর কাঠামো

Folder-এর কাঠামো

```
task-manager/
├── index.html          Page-এর গঠন
├── style.css           চেহারা
├── task-core.js        তথ্যের নিয়ম; DOM নেই (Browser ও Node.js)
├── app.js              DOM, Event ও Storage
├── README.md
└── tests/
    ├── load-script.js
    ├── task-core.test.js   Node.js-এ Unit Test
    ├── cdp.mjs
    └── browser-check.mjs   আসল Chrome-এ Page-এর Test
```

কাজের প্রবাহ: ব্যবহারকারীর কাজ থেকে একটি Event; Listener `task-core.js`-এর একটি Function দিয়ে নতুন Task-তালিকা তৈরি করে; সেই তালিকা State-এ বসে, `localStorage`-এ সংরক্ষিত হয়, এবং `render()` পর্দা নতুন করে আঁকে।

index.html

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Task Manager</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>Task Manager</h1>

    <form id="task-form" novalidate>
      <label for="task-input">নতুন Task</label>
      <div class="row">
        <input id="task-input" autocomplete="off">
```

```
        <button type="submit">যোগ করুন</button>
      </div>
    </form>
    <p id="notice" role="status" aria-live="polite"></p>

    <div class="tools">
      <label>খুঁজুন <input id="search" type="search"></label>
      <label>দেখান
        <select id="filter">
          <option value="all">সব</option>
          <option value="active">বাকি</option>
          <option value="done">সম্পন্ন</option>
        </select>
      </label>
      <button id="clear-done" type="button">সম্পন্নগুলো সরান</button>
    </div>

    <p id="summary"></p>
    <ul id="task-list"></ul>
    <p id="empty">কোনো Task নেই।</p>
  </main>

  <script src="task-core.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

দুটি Script `<body>`-র শেষে, এবং ক্রম গুরুত্বপূর্ণ: `app.js` যে Function ব্যবহার করে, সেগুলো `task-core.js`-এ, তাই সেটি আগে। Field-এ `maxlength` দেওয়া হয়নি ইচ্ছা করে: সীমা পার হলে ব্যবহারকারী চুপচাপ থেমে যাওয়ার বদলে একটি ব্যাখ্যা পান।

style.css

চেহারার File দুই অংশে; দুটি মিলে সম্পূর্ণ File।

style.css · CSS অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
```

```
  max-width: 640px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

.row,
.tools {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

/* ... */
```

style.css · CSS অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
/* ... */

input,
select,
button {
  font: inherit;
  padding: 6px 10px;
}

#task-input,
#task-list .title,
#task-list .edit-input {
  flex: 1;
}

#task-list li {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px 0;
  border-bottom: 1px solid #e3e7ef;
}

#task-list li.done .title {
  text-decoration: line-through;
  color: #6b7585;
}

#notice {
  min-height: 1.5em;
```

```
  color: #12305f;
}
```

সম্পন্ন Task-এর চেহারা CSS-এ `li.done .title` নিয়মে লেখা; JavaScript শুধু `done` Class যোগ বা সরায় (অনুচ্ছেদ ৯.৩)।

task-core.js

তথ্যের সব নিয়ম এখানে। এই File-এ `document` বা `localStorage`-এর কোনো উল্লেখ নেই, তাই একই File Browser-এ চলে, আবার Node.js-এর Test-এও। দুই অংশ মিলে সম্পূর্ণ File।

task-core.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
'use strict';

const TITLE_LIMIT = 120;

// নাম যাচাই: সমস্যা থাকলে বার্তা, না থাকলে null।
function checkTitle(text) {
  const title = String(text ?? '').trim();

  if (title === '') {
    return 'Task-এর একটি নাম লিখুন।';
  }
  if (title.length > TITLE_LIMIT) {
    return `নাম ${TITLE_LIMIT} অক্ষরের বেশি হতে পারবে না।`;
  }
  return null;
}

function addTask(tasks, text, id) {
  const error = checkTitle(text);

  if (error) {
    return { tasks, error };
  }

  const task = { id: String(id), title: text.trim(), done: false };
  return { tasks: [...tasks, task], error: null };
}

function renameTask(tasks, id, text) {
  const error = checkTitle(text);

  if (error) {
    return { tasks, error };
  }

  const next = tasks.map((task) =>
```

```
    task.id === id ? { ...task, title: text.trim() } : task);
  return { tasks: next, error: null };
}

// ...
```

task-core.js · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

const toggleTask = (tasks, id) =>
  tasks.map((task) => (task.id === id ? { ...task, done: !task.done } : task));

const deleteTask = (tasks, id) => tasks.filter((task) => task.id !== id);

const clearDone = (tasks) => tasks.filter((task) => !task.done);

function visibleTasks(tasks, filter = 'all', query = '') {
  const needle = query.trim().toLowerCase();

  return tasks.filter((task) => {
    const statusOk = filter === 'all' || (filter === 'done') === task.done;
    return statusOk && task.title.toLowerCase().includes(needle);
  });
}

function countTasks(tasks) {
  const done = tasks.filter((task) => task.done).length;
  return { total: tasks.length, active: tasks.length - done, done };
}

// localStorage-এর লেখা থেকে নিরাপদে Task-এর তালিকা; ভাঙা হলে ফাঁকা তালিকা।
function parseStored(raw) {
  try {
    const data = JSON.parse(raw ?? '[]');
    return Array.isArray(data) ? data.filter(isTask) : [];
  } catch {
    return [];
  }
}

function isTask(value) {
  return typeof value?.id === 'string' &&
    typeof value.title === 'string' &&
    checkTitle(value.title) === null &&
```

```
    typeof value.done === 'boolean';
}
```

- **অধ্যায় ৭-এর ধরন:** প্রতিটি Function মূল তালিকা না বদলে নতুন তালিকা ফেরত দেয়; ব্যর্থ হলে মূল তালিকা এবং একটি বার্তা।
- **`visibleTasks`-এর ছাঁকনি:** `filter === 'all'` হলে সব; নইলে `(filter === 'done') === task.done`। "সম্পন্ন" বাছলে বাম পাশ `true`, তাই শুধু সম্পন্ন Task মেলে; "বাকি" বাছলে বাম পাশ `false`, তাই শুধু বাকিগুলো। দুটি `if`-এর কাজ একটি তুলনায়।
- **`parseStored` ও `isTask`:** অনুচ্ছেদ ৯.১০-এর নিরাপদ পড়া, এক ধাপ এগিয়ে। শুধু Array কি না নয়, প্রতিটি Task-এর গঠনও যাচাই করা হয়; অচেনা বা ভাঙা Task বাদ পড়ে, বাকিগুলো থাকে।

app.js

Page-এর সঙ্গে সব যোগাযোগ এখানে। চার অংশ মিলে সম্পূর্ণ File।

অংশ ১: Element, State এবং সংরক্ষণ

app.js · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Browser

```
'use strict';

const STORAGE_KEY = 'jsm-ch09-tasks';

const form = document.querySelector('#task-form');
const input = document.querySelector('#task-input');
const list = document.querySelector('#task-list');
const notice = document.querySelector('#notice');
const summary = document.querySelector('#summary');
const search = document.querySelector('#search');
const filter = document.querySelector('#filter');
const empty = document.querySelector('#empty');

let tasks = parseStored(readStorage());
let editingId = null;

function readStorage() {
  try {
    return localStorage.getItem(STORAGE_KEY);
  } catch {
    return null;
  }
}

function save() {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
  } catch {
```

```
    notice.textContent = 'সংরক্ষণ করা যায়নি; Page বন্ধ করলে পরিবর্তন হারাবে।';
  }
}

// প্রতিটি পরিবর্তন এই পথে: State বদলানো, সংরক্ষণ, নতুন করে আঁকা।
function update(next, message = '') {
  tasks = next;
  notice.textContent = message;
  save();
  render();
}

// ...
```

State দুটি Variable: `tasks` (তালিকা) এবং `editingId` (কোন Task-এর নাম এখন বদলানো হচ্ছে)। সব পরিবর্তন `update` দিয়ে যায়, তাই সংরক্ষণ বা আঁকা কোথাও ভুলে বাদ পড়ার সুযোগ নেই।

অংশ ২: আঁকা

app.js · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Browser

```
// ...

function render() {
  const shown = visibleTasks(tasks, filter.value, search.value);
  const count = countTasks(tasks);

  summary.textContent =
    `মোট ${count.total} · বাকি ${count.active} · সম্পন্ন ${count.done}`;
  empty.hidden = shown.length > 0;
  list.replaceChildren(...shown.map(taskRow));
}

function taskRow(task) {
  const row = document.createElement('li');
  row.dataset.id = task.id;
  row.classList.toggle('done', task.done);

  if (task.id === editingId) {
    const field = document.createElement('input');
    field.className = 'edit-input';
    field.value = task.title;
    field.setAttribute('aria-label', 'Task-এর নতুন নাম');
    row.append(field, button('সংরক্ষণ', 'save'), button('বাতিল', 'cancel'));
    return row;
  }

  const check = document.createElement('input');
  check.type = 'checkbox';
```

```
  check.checked = task.done;
  check.dataset.action = 'toggle';
  check.setAttribute('aria-label', `সম্পন্ন: ${task.title}`);

  const title = document.createElement('span');
  title.className = 'title';
  title.textContent = task.title;

  row.append(check, title, button('বদলান', 'edit'), button('মুছুন', 'delete'));
  return row;
}

function button(label, action) {
  const element = document.createElement('button');
  element.type = 'button';
  element.textContent = label;
  element.dataset.action = action;
  return element;
}

// ...
```

`render` প্রতিবার পুরো তালিকা নতুন করে তৈরি করে। কয়েকশো Task পর্যন্ত এটি চোখে পড়ার মতো ধীর নয়, এবং Code সরল থাকে। প্রতিটি সারিতে Task-এর `id` আছে `data-id`-তে, আর প্রতিটি Button-এ তার কাজের নাম `data-action`-এ; পরের অংশের একটি Listener এই দুটি দেখেই সব কাজ চেনে। Checkbox-এর `aria-label` Screen Reader-কে বলে কোন Task-এর Checkbox।

অংশ ৩: যোগ করা এবং নাম বদলানো

app.js · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Browser

```
// ...

const newId = () => `${Date.now()}-${Math.random().toString(16).slice(2, 8)}`;

form.addEventListener('submit', (event) => {
  event.preventDefault();
  const result = addTask(tasks, input.value, newId());

  if (result.error) {
    notice.textContent = result.error;
    input.focus();
    return;
  }

  form.reset();
  update(result.tasks, 'Task যোগ হয়েছে।');
});
```

```
function saveEdit(id) {
  const field = list.querySelector('.edit-input');
  const result = renameTask(tasks, id, field.value);

  if (result.error) {
    notice.textContent = result.error;
    field.focus();
    return;
  }

  editingId = null;
  update(result.tasks, 'নাম বদলানো হয়েছে।');
}

function startEdit(id) {
  editingId = id;
  render();
  list.querySelector('.edit-input').focus();
}

function cancelEdit() {
  editingId = null;
  render();
}

// ...
```

অংশ ৪: তালিকার Event এবং শুরু

app.js · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Browser

```
// ...

// Event Delegation: তালিকার একটি Listener সব সারির সব Button সামলায়।
list.addEventListener('click', (event) => {
  const target = event.target.closest('[data-action]');

  if (!target) {
    return;
  }

  const id = target.closest('li').dataset.id;
  const action = target.dataset.action;

  if (action === 'toggle') update(toggleTask(tasks, id));
  if (action === 'delete') update(deleteTask(tasks, id), 'Task মুছে ফেলা হয়েছে।');
  if (action === 'edit') startEdit(id);
  if (action === 'save') saveEdit(id);
```

```
  if (action === 'cancel') cancelEdit();
});

list.addEventListener('keydown', (event) => {
  if (!event.target.matches('.edit-input')) {
    return;
  }
  if (event.key === 'Enter') saveEdit(event.target.closest('li').dataset.id);
  if (event.key === 'Escape') cancelEdit();
});

search.addEventListener('input', render);
filter.addEventListener('change', render);
document.querySelector('#clear-done').addEventListener('click', () =>
  update(clearDone(tasks), 'সম্পন্ন Task সরানো হয়েছে।'));

render();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **যোগ করা।** `submit` Event Button-এ Click ও Enter—দুটোই ধরে। `addTask` ব্যর্থ হলে বার্তা দেখিয়ে Focus Field-এ ফেরত যায়, যাতে ব্যবহারকারী সঙ্গে সঙ্গে ঠিক করতে পারেন। সফল হলে Form খালি হয় এবং `update` বাকিটা করে।
- **একটি Listener, পাঁচটি কাজ।** তালিকার `click` Listener `closest('[data-action]')` দিয়ে কাজের Element খোঁজে, `closest('li')` দিয়ে সারি, আর `dataset.id` দিয়ে Task। Checkbox-এর Click-ও এখানে আসে (`toggle`)। `render` প্রতিবার সারিগুলো নতুন তৈরি করলেও Listener একবারই লাগানো হয়েছে।
- **নাম বদলানো।** `startEdit` শুধু State-এ `editingId` বসিয়ে `render` ডাকে; `taskRow` সেই সারির জন্য Checkbox-এর বদলে একটি Field আঁকে। এরপর Enter বা "সংরক্ষণ" দিয়ে `saveEdit`, Escape বা "বাতিল" দিয়ে `cancelEdit`। নতুন নাম ফাঁকা হলে `renameTask` বার্তা দেয় এবং Field খোলা থাকে।
- **খোঁজা ও ছাঁকা।** এগুলো State বদলায় না, শুধু কোন অংশ দেখানো হবে তা বদলায়, তাই সরাসরি `render`।
- **সংরক্ষণ।** `update` প্রতিটি পরিবর্তনের পর পুরো তালিকা সংরক্ষণ করে; Page খোলার সময় `parseStored(readStorage())` তা ফিরিয়ে আনে।
- **নিরাপত্তা।** Task-এর নাম শুধু `textContent` এবং Attribute হিসেবে বসানো হয়; কোথাও `innerHTML` নেই।

চালানো এবং প্রত্যাশিত আচরণ

`index.html` Browser-এ খুলুন (File-টিতে Double Click করলেই হবে)। Unit Test চালাতে Project Folder থেকে:

Terminal

```
node --test
node tests/browser-check.mjs
```

কাজ	যা হওয়ার কথা
ফাঁকা রেখে "যোগ করুন"	বার্তা "Task-এর একটি নাম লিখুন।"; কিছু যোগ হয় না
১২১ অক্ষরের নাম	বার্তা "নাম 120 অক্ষরের বেশি হতে পারবে না।"
নাম লিখে Enter	Task যোগ; Field খালি; সংখ্যা হালনাগাদ
Checkbox	নাম কেটে দেওয়া চেহারা; "বাকি" ও "সম্পন্ন" সংখ্যা বদলায়
"বদলান", নতুন নাম, Enter	নাম বদলায়; Escape দিলে আগের নাম থাকে
নামে `<img src="" onerror="...">`	লেখাটি হুবহু দেখা যায়; কোনো Code চলে না
Page Reload	সব Task ও তাদের অবস্থা ফিরে আসে

Companion Source Code-এ দুই ধরনের Automated Test আছে। `task-core.test.js` Node.js-এ তথ্যের নিয়মগুলোর ১৬টি Test চালায়। `browser-check.mjs` আসল Chrome খুলে Page-এ লিখে, Click করে ও Key চেপে ওপরের সারণির সব আচরণসহ ১৭টি আচরণ যাচাই করে; এর জন্য কম্পিউটারে Chrome বা Edge থাকতে হবে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`addTask is not defined`	`task-core.js` লোড হয়নি বা `app.js` -এর পরে	Script-এর ক্রম ও File-এর নাম
যোগ করলে Page Reload হয়	`preventDefault` নেই	Submit Handler-এর প্রথম লাইন
নতুন সারির Button কাজ করে না	Listener সারিতে, তালিকায় নয়	Event Delegation
Reload-এ তালিকা হারায়	`save` ডাকা হয়নি, বা Key বদলেছে	সব পরিবর্তন `update` দিয়ে; DevTools-এর Application
Page ফাঁকা, Console-এ JSON Error	সংরক্ষিত তথ্য ভাঙা এবং যাচাই নেই	`parseStored`
খুঁজলে কিছুই দেখায় না	ছাঁকনি ও খোঁজার শব্দ একসঙ্গে প্রযোজ্য	"সব" বেছে আবার দেখুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য শুধু এই Browser-এ থাকে; অন্য Device-এ দেখা যায় না, এবং Browser-এর তথ্য মুছলে হারায়। একাধিক Tab-এ একসঙ্গে খুললে শেষ সংরক্ষণটি টেকে। `file://` থেকে খুললে কিছু Browser Storage-এর আচরণ বদলায়; নির্ভরযোগ্য পরীক্ষার জন্য একটি Local Server ব্যবহার করুন (README দেখুন)। Server-এর সঙ্গে তথ্য আদান-প্রদান অধ্যায় ১০-এর বিষয়।

৯.১৫ অনুশীলনী

অনুশীলনগুলো একটি ফাঁকা Page-এ চলে: যে Element দরকার, Script নিজে তৈরি করে নেয়। একটি HTML File-এ শুধু `<script src="solution.js"></script>` রেখে `<body>`-র ভেতরে চালান।

প্রাথমিক স্তর

১. একটি `h1` তৈরি করে Page-এ যোগ করুন, তারপর `querySelector` দিয়ে খুঁজে তার লেখা বদলান এবং নতুন লেখাটি Console-এ দেখান।

২. "থিম বদলান" নামে একটি Button তৈরি করুন, যা চাপলে `body`-তে `dark` Class যোগ বা সরায়, এবং প্রতিবার Console-এ `Dark` বা `Light` দেখায়।

৩. `button.addEventListener('click', sayHello())` কেন কাজ করে না? ঠিক করুন।

৪. অনুচ্ছেদ ৯.৬-এর Code-এ `like`-এর Listener-এ `event.stopPropagation()` যোগ করলে "পছন্দ" Button-এ Click-এ কী কী ছাপা হবে?

৫. `['আম', 'কাঁঠাল', 'লিচু']` থেকে `createElement` দিয়ে একটি তালিকা তৈরি করুন; `innerHTML` ব্যবহার করবেন না।

মধ্যম স্তর

৬. একটি লেখার Box এবং তার নিচে "ব্যবহৃত/১২০" গণনা তৈরি করুন, যা প্রতিবার লেখার সময় হালনাগাদ হয়; সীমা পার হলে `over` Class।

৭. তিনটি সারির একটি তালিকা তৈরি করুন, প্রতিটিতে একটি "মুছুন" Button। একটিমাত্র Listener দিয়ে Button-এর সারিটি মুছুন এবং বাকি সংখ্যা দেখান।

৮. একটি লেখার Field-এ Enter চাপলে লেখাটি একটি তালিকায় যোগ হবে এবং Field খালি হবে; ফাঁকা হলে যোগ হবে না।

৯. `loadTheme` ও `saveTheme` লিখুন, যা `localStorage`-এ থিম রাখে। সংরক্ষিত মান `'dark'` ছাড়া অন্য কিছু হলে, বা Storage কাজ না করলে, `'light'` ধরুন।

চ্যালেঞ্জ স্তর

১০. একটি খোঁজার Field তৈরি করুন: লেখার সময় Console-এ খোঁজার শব্দ দেখাবে, আর Escape চাপলে Field খালি করবে।

১১. Task Manager-এ "সব সম্পন্ন" Button যোগ করতে প্রথমে `task-core.js`-এর জন্য একটি Pure Function `completeAll(tasks)` লিখুন, এবং দেখান যে মূল তালিকা বদলায়নি।

১২. মোছার পর "ফিরিয়ে আনুন" সুবিধার জন্য `deleteWithUndo(tasks, id)` লিখুন, যা নতুন তালিকা এবং একটি `undo` Function ফেরত দেয়; `undo` Task-টিকে তার আগের জায়গায় ফিরিয়ে আনবে।

৯.১৬ অনুশীলনীর সমাধান

সমাধান ১

solution.js · JavaScript চলবে: Browser

```
const heading = document.createElement('h1');
heading.textContent = 'পুরোনো শিরোনাম';
document.body.append(heading);

document.querySelector('h1').textContent = 'নতুন শিরোনাম';
console.log(document.querySelector('h1').textContent);
```

Output

```
নতুন শিরোনাম
```

সমাধান ২

solution.js · JavaScript চলবে: Browser

```
const toggle = document.createElement('button');
toggle.id = 'theme';
toggle.type = 'button';
toggle.textContent = 'থিম বদলান';
document.body.append(toggle);

toggle.addEventListener('click', () => {
  const isDark = document.body.classList.toggle('dark');
  console.log(isDark ? 'Dark' : 'Light');
});
```

দুবার চাপলে:

Output

```
Dark
Light
```

`classList.toggle` জানায় Class এখন আছে কি না, তাই আলাদা করে `contains` লাগেনি।

সমাধান ৩ — `sayHello()` লেখার সঙ্গে সঙ্গে Function চলে যায়, এবং তার ফেরত মান `undefined` Listener হিসেবে দেওয়া হয়। Function-টি নিজে দিতে হবে, Call করে নয়: `button.addEventListener('click', sayHello)`।

সমাধান ৪ — শুধু `like: click`। Event Button-এ থেমে যায়, তাই `card` ও `document`-এর Listener চলে না। শিরোনামে Click করলে আগের মতোই `card: target H2` এবং `document`।

সমাধান ৫

solution.js · JavaScript — চলবে: Browser

```
const fruits = ['আম', 'কাঁঠাল', 'লিচু'];
const list = document.createElement('ul');

const items = fruits.map((fruit) => {
  const item = document.createElement('li');
  item.textContent = fruit;
  return item;
});

list.append(...items);
document.body.append(list);

console.log(list.children.length);
console.log(list.lastElementChild.textContent);
```

Output

```
3
লিচু
```

সমাধান ৬

solution.js · JavaScript — চলবে: Browser

```
const LIMIT = 120;
const field = document.createElement('textarea');
const counter = document.createElement('p');
field.id = 'note';
document.body.append(field, counter);

function updateCounter() {
  const used = field.value.length;
  counter.textContent = `${used}/${LIMIT}`;
  counter.classList.toggle('over', used > LIMIT);
  console.log(counter.textContent);
}

field.addEventListener('input', updateCounter);
updateCounter();
```

Page খোলার পর "হ্যালো JavaScript" লিখলে:

Output

```
0/120
17/120
```

"হ্যালো" চোখে দুটি অক্ষর (হ্যা, লো), কিন্তু `length` ছয় (অধ্যায় ৮)। Task Manager-এর সীমাও `length` দিয়ে, যা সংরক্ষণের সীমার জন্য উপযুক্ত।

সমাধান ৭

solution.js · JavaScript চলবে: Browser

```
const list = document.createElement('ul');

for (const name of ['ক', 'খ', 'গ']) {
  const row = document.createElement('li');
  const removeButton = document.createElement('button');
  removeButton.type = 'button';
  removeButton.dataset.name = name;
  removeButton.textContent = `মুছুন ${name}`;
  row.append(removeButton);
  list.append(row);
}
document.body.append(list);

list.addEventListener('click', (event) => {
  const button = event.target.closest('button');

  if (!button) {
    return;
  }

  button.closest('li').remove();
  console.log(`${button.dataset.name} মুছে গেছে; বাকি ${list.children.length}`);
});
```

"মুছুন খ" চাপলে:

Output

```
খ মুছে গেছে; বাকি 2
```

সমাধান ৮

solution.js · JavaScript — চলবে: Browser

```
const input = document.createElement('input');
const list = document.createElement('ul');
input.id = 'item';
document.body.append(input, list);

input.addEventListener('keydown', (event) => {
  if (event.key !== 'Enter') {
    return;
  }

  const text = input.value.trim();

  if (text === '') {
    console.log('ফাঁকা; যোগ হলো না');
    return;
  }

  const item = document.createElement('li');
  item.textContent = text;
  list.append(item);
  input.value = '';
  console.log(`যোগ হলো: ${text}; মোট ${list.children.length}`);
});
```

শুধু ফাঁকা জায়গা লিখে Enter, তারপর "দুধ কেনা" লিখে Enter:

Output

```
ফাঁকা; যোগ হলো না
যোগ হলো: দুধ কেনা; মোট 1
```

এখানে `<form>` ছাড়াই `keydown` দিয়ে কাজ হয়েছে। Project-এ `<form>` ও `submit` ব্যবহার করা হয়েছে, কারণ তাতে Button ও Enter দুটোই বিনা বাড়তি Code-এ কাজ করে।

সমাধান ৯

solution.js · JavaScript — চলবে: Browser

```
const THEME_KEY = 'jsm-theme';

function loadTheme() {
  try {
    return localStorage.getItem(THEME_KEY) === 'dark' ? 'dark' : 'light';
```

```
  } catch {
    return 'light';
  }
}

function saveTheme(theme) {
  try {
    localStorage.setItem(THEME_KEY, theme);
  } catch {
    console.log('থিম সংরক্ষণ করা গেল না');
  }
}

console.log(loadTheme());
saveTheme('dark');
console.log(loadTheme());
localStorage.setItem(THEME_KEY, 'purple');
console.log(loadTheme());
localStorage.removeItem(THEME_KEY);
```

Output

```
light
dark
light
```

Page খোলার সময় `document.body.classList.toggle('dark', loadTheme() === 'dark')` লিখলে সংরক্ষিত থিম প্রয়োগ হয়।

সমাধান ১০

solution.js · JavaScript — চলবে: Browser

```
const search = document.createElement('input');
search.id = 'search';
document.body.append(search);

search.addEventListener('input', () => {
  console.log(`খুঁজছেন: ${search.value}`);
});

search.addEventListener('keydown', (event) => {
  if (event.key === 'Escape') {
    search.value = '';
    console.log('খোঁজা মুছে ফেলা হলো');
  }
});
```

"JavaScript" লিখে Escape চাপলে:

Output

```
খুঁজছেন: JavaScript
খোঁজা মুছে ফেলা হলো
```

Code দিয়ে `value` বদলালে `input` Event ঘটে না; তাই Escape-এর পর "খুঁজছেন" লাইনটি আবার আসেনি। Task Manager-এ এমন ক্ষেত্রে মান বদলানোর পর নিজে `render()` ডাকতে হবে।

সমাধান ১১

complete-all.js · JavaScript — চলবে: Browser Console ও Node.js

```
const completeAll = (tasks) => tasks.map((task) => ({ ...task, done: true }));

const tasks = [
  { id: '1', title: 'বাজার', done: false },
  { id: '2', title: 'বই', done: true },
];
const next = completeAll(tasks);

console.log(next.map((task) => task.done).join(', '));
console.log(tasks[0].done);
console.log(next[0] === tasks[0]);
```

Output

```
true, true
false
false
```

Pure Function বলে এটি Browser ছাড়াই পরীক্ষা করা গেল। Page-এ যোগ করতে একটি Button এবং তার Listener-এ শুধু `update(completeAll(tasks), 'সব সম্পন্ন।')`।

সমাধান ১২

undo.js · JavaScript — চলবে: Browser Console ও Node.js

```
function deleteWithUndo(tasks, id) {
  const index = tasks.findIndex((task) => task.id === id);

  if (index === -1) {
    return { tasks, undo: null };
  }
```

```
  const removed = tasks[index];
  const undo = (current) => [
    ...current.slice(0, index),
    removed,
    ...current.slice(index),
  ];

  return { tasks: tasks.filter((task) => task.id !== id), undo };
}

const start = [
  { id: 'a', title: 'ক' },
  { id: 'b', title: 'খ' },
  { id: 'c', title: 'গ' },
];
const { tasks, undo } = deleteWithUndo(start, 'b');

console.log(tasks.map((task) => task.title).join(''));
console.log(undo(tasks).map((task) => task.title).join(''));
console.log(deleteWithUndo(start, 'x').undo);
```

Output

```
কগ
কখগ
null
```

`undo` একটি Closure (অধ্যায় ৫): মুছে ফেলা Task এবং তার অবস্থান মনে রাখে। এটি বর্তমান তালিকা নেয়, তাই মোছার পর অন্য পরিবর্তন হয়ে থাকলেও সেগুলো হারায় না।

৯.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কিছু না মিললে `document.querySelector('#missing')` কী দেয়?

ক) `undefined`

খ) একটি Error

গ) `null`

ঘ) একটি ফাঁকা `NodeList`

প্রশ্ন ২. `<head>`-এ রাখা Script যেন পুরো HTML পড়ার পরে চলে, সে জন্য কী লিখবেন?

ক) `defer`

খ) `async`

গ) `type="module"` ছাড়া উপায় নেই

ঘ) `onload`

প্রশ্ন ৩. ব্যবহারকারীর লেখা একটি Element-এ নিরাপদে বসানোর উপায়?

ক) `innerHTML`

খ) `outerHTML`

গ) `insertAdjacentHTML`

ঘ) `textContent`

প্রশ্ন ৪. `button.addEventListener('click', save())`-এর সমস্যা কী?

ক) `click` ভুল Event-এর নাম

খ) `save` এখনই চলে; Listener হিসেবে যায় তার ফেরত মান

গ) Arrow Function বাধ্যতামূলক

ঘ) কোনো সমস্যা নেই

প্রশ্ন ৫. Child Element-এর Click Parent-এর Listener-এও পৌঁছায়। এই আচরণের নাম?

ক) Bubbling

খ) Hoisting

গ) Delegation

ঘ) Rendering

প্রশ্ন ৬. Event Delegation-এর প্রধান সুবিধা কোনটি?

ক) Event দ্রুত ঘটে

খ) `preventDefault` লাগে না

গ) একটি Listener পরে তৈরি হওয়া Element-ও সামলায়

ঘ) `event.target` লাগে না

প্রশ্ন ৭. Submit Handler-এ `event.preventDefault()` কী থামায়?

ক) Event-এর ওপরে ওঠা

খ) Form পাঠিয়ে Page নতুন করে Load হওয়া

গ) অন্য সব Listener

ঘ) `input` Event

প্রশ্ন ৮. `localStorage.setItem('n', 5)`-এর পর `typeof localStorage.getItem('n')`?

ক) `'number'`

খ) `'object'`

গ) `'undefined'`

ঘ) `'string'`

প্রশ্ন ৯. Task Manager-এ তথ্যের নিয়ম আলাদা `task-core.js`-এ রাখার মূল কারণ?

ক) Browser দ্রুত চলে

খ) DOM ছাড়া Node.js-এ Unit Test করা যায়

গ) `localStorage` শুধু আলাদা File থেকে কাজ করে

ঘ) HTML-এ একটিমাত্র Script রাখা যায় না

প্রশ্ন ১০. `document.querySelectorAll('li').map(...)` কেন Error দেয়?

ক) `NodeList`-এ `map` নেই

খ) `li` ভুল Selector

গ) `querySelectorAll` সব সময় `null` দেয়

ঘ) Arrow Function চলে না

৯.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–ক · ৬–গ · ৭–খ · ৮–ঘ · ৯–খ · ১০–ক

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	একক Element না পেলে `null`; `querySelectorAll` দেয় ফাঁকা `NodeList`।
২	ক	`defer` File আগে আনে, কিন্তু চালায় HTML পড়া শেষে, লেখার ক্রমে।
৩	ঘ	`textContent` লেখাকে HTML হিসেবে পড়ে না; বাকিগুলো পড়ে।
৪	খ	Parenthesis Function Call করে; Listener-এ Function নিজে দিতে হয়।
৫	ক	Event DOM গাছ বেয়ে `document` পর্যন্ত ওপরে ওঠে।
৬	গ	Listener Parent-এ থাকে, তাই পরে যোগ হওয়া সন্তানের Event-ও পায়।
৭	খ	Browser-এর নিজস্ব আচরণ থামায়; Propagation থামায় `stopPropagation`।
৮	ঘ	`localStorage` সব কিছু লেখা হিসেবে রাখে।
৯	খ	Pure Function-এ DOM নেই, তাই Browser ছাড়াই পরীক্ষা করা যায়।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	ক	`NodeList` Array নয়; `[...list]` বা `Array.from(list)` দিয়ে আগে Array বানাতে হয়।

৯.১৯ অধ্যায়ের সারসংক্ষেপ

- DOM হলো Page-এর Element Object-এর গাছ; JavaScript এটি পড়ে ও বদলায়, Browser পর্দা হালনাগাদ করে।
- Script Element পড়ার আগে চললে `null`; Script `<body>`-র শেষে বা `defer`।
- খোঁজা `querySelector` / `querySelectorAll`; নিজের তথ্য `data-` Attribute ও `dataset`।
- বদলানো: `textContent`, `classList`, `setAttribute`, `hidden`; চেহারা CSS Class-এ।
- তৈরি `createElement`, যোগ `append` / `prepend`, সরানো `remove`, সব বদলানো `replaceChildren`।
- `addEventListener`-এ Function নিজে দিন; Event Object-এ `type`, `target`, `key`।
- Event Bubbling-এর কারণে Parent-এ একটি Listener দিয়ে Event Delegation; `closest` দিয়ে আসল Element।
- Form-এ `submit` এবং প্রথমেই `preventDefault`; Label, `aria-live`, Keyboard-এর ব্যবহারযোগ্যতা।
- এক State, এক `render`; তথ্যের নিয়ম DOM-মুক্ত Pure Function-এ।
- `localStorage` শুধু লেখা রাখে; পড়ার সময় যাচাই, ভাঙলে নিরাপদ মান; সংবেদনশীল তথ্য নয়।

পরের অধ্যায়ে Asynchronous JavaScript: অপেক্ষা করতে হয় এমন কাজ, যেমন Server থেকে তথ্য আনা, Page আটকে না দিয়ে কীভাবে করা যায়।

৯.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
DOM	Page-এর Element Object-গুলোর গাছ।
Selector	CSS-এর নিয়মে Element চেনার লেখা: `#id`, `.class`, `tag`।
NodeList	`querySelectorAll`-এর ফল; Array-র মতো, কিন্তু Array নয়।
`defer`	HTML পড়া শেষে Script চালানোর নির্দেশ।
Event	Browser-এ ঘটা একটি ঘটনা: Click, লেখা, Submit।
Event Listener	Event ঘটলে চলার জন্য দেওয়া Function।
Event Object	Listener-এ আসা Event-এর তথ্য।
Bubbling	Event-এর DOM গাছ বেয়ে ওপরে ওঠা।

পরিভাষা	অর্থ
Event Delegation	Parent-এ একটি Listener দিয়ে সন্তানদের Event সামলানো।
State	Application-এর বর্তমান তথ্য, যা থেকে Page আঁকা হয়।
Render	State থেকে Page-এর অংশ তৈরি করা।
`aria-live`	লেখা বদলালে Screen Reader-কে তা পড়তে বলার Attribute।

৯.২১ আরও পড়ুন

- **MDN — Introduction to the DOM:** `https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction` — DOM-এর মূল ধারণা।
- **MDN — Introduction to events:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events` — Event, Bubbling এবং Delegation।
- **MDN — Event bubbling:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Event_bubbling` — Propagation-এর বিস্তারিত।
- **MDN — Window.localStorage:** `https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage` — Storage-এর নিয়ম ও সীমা।
- **web.dev — Learn Accessibility:** `https://web.dev/learn/accessibility` — সবার জন্য ব্যবহারযোগ্য Web Page।